Lean proof verification: desk syllabus

SHAD Fall 2026 + Mathematics in Lean (MIL) + Theorem Proving in Lean 4 (TPiL)

For a Lean beginner with 3-4 hours/week: 28 study weeks in 14 two-week units. **Start:** 26 October 2026. **Final review:** 21 May 2027. A winter break, 21 December-3 January, is built into these 30 calendar weeks. All dates are suggested Friday deadlines at 5 p.m. America/Chicago. This covers the repository snapshot of 21 October, not unreleased material.

1. Route and weekly deliverables

Wks	Due	Study focus	Deliverable
1-2	Nov 6	Setup; propositional logic	Hello + Propositional homework
3-4	Nov 20	Quantifiers; equality; rewriting	Quantifiers homework
5-6	Dec 4	Ordinary and strong induction	Induction homework; checkpoint A
7-8	Dec 18	Functions; types; lambdas	Functions homework, including counterexamples
9-10	Jan 15	Sets; extensionality; images	Set homework, including convexity
11-12	Jan 29	Inductive data; recursion	BinTree homework, including the definition of add
13-14	Feb 12	Structures; instances; hierarchies	Small class/instance example; checkpoint B
15-16	Feb 26	Filters; limits; continuity	Two new limit proofs; explain their hypotheses
17-18	Mar 12	Finsets; finite sums; series	Both SqrtSum theorems
19-20	Mar 26	Factorials; casts; series bounds	First five IrrationalE helper theorems
21-22	Apr 9	Irrationality of e; derivatives	Final IrrationalE theorem; checkpoint C
23-24	Apr 23	Newton: plan and auxiliary lemmas	Exact paper proof + at least two checked lemmas
25-26	May 7	Newton; measure/integration survey	Newton complete; integral and a.e. examples
27-28	May 21	Independent project; proof audit	Small formalization + replay; checkpoint D

Every week: Produce a small checked result in each unit's first week, then finish and reconstruct in its second week. README.md lists the intermediate milestones; Progress.md has all 28 weekly deadlines. The core includes all ten non-bonus homework files. Shift dates if needed; keep the prerequisite order.

Book order: MIL 1, 3 and selected 2 alongside logic; 5.2/5.4 for induction; 4.2 then 4.1 for functions/sets; 6.3 and 7-8 for data/structures; 3.6 and 11 for limits; 6.1/6.2 for sums; 5.1 for divisibility practice; 12 then 13 for analysis. TPiL 1-5 builds foundations, 6 supports library use, 7-10 explains data and instances, and 11-12 completes the language/foundations review. The full week-by-week crosswalk is in README.md.

Extension: Study weeks 29-32: Cantor, then Cantor-Bernstein; MIL 4.3 after an independent attempt. Later: MIL 9-10 and deeper topology/analysis. Full mastery of these extensions is outside the core timeline.

2. Concept checklist

Mark an item only after you can explain it and demonstrate it in a checked example. **Core** means working fluency; **survey** means recognize the object and read a representative theorem. A survey item can become a later project.

Foundations and elementary proofs - core, study weeks 1-8

- ☐ Read the goal and local context; distinguish data, propositions, and proofs.
- ☐ Explain proofs as terms, implication as a function, and universal quantification as a dependent function.
- ☐ Distinguish `Prop`, `Bool`, Type, `Sort`, and universe levels at a basic level.
- ☐ Use lambdas, currying, partial application, explicit/implicit arguments, and namespaces.
- ☐ Construct/use `True`, `False`, negation, conjunction, disjunction, equivalence, and quantifiers.
- ☐ Distinguish constructive reasoning from an application of classical reasoning.
- ☐ Use `intro`, `exact`, `apply`, have, constructors, case analysis, and explicit witnesses.
- ☐ Distinguish definitional equality, propositional equality, and equivalence; use `rfl`, `rw`, `simp`, and `calc`.
- ☐ Prove ordinary/strong induction statements; generalize the induction hypothesis when needed.

Mathematical objects and programs - core, study weeks 7-14

- ☐ Define functions; prove injectivity/surjectivity; build and verify a counterexample.
- ☐ View sets as predicates; use membership, subset, union/intersection, images, and preimages.
- ☐ Use function and set extensionality; track coercions between naturals, integers, and reals.
- ☐ Define structures and inductive types; use constructors, projections, and pattern matching.
- ☐ Define structurally recursive functions and prove a structural invariant.
- ☐ Explain termination, an indexed inductive family, and why strict positivity matters.
- ☐ Use classes, instances, inherited structures, and instance synthesis; read theorem assumptions.
- ☐ Distinguish `Set`, `Finset`, `Fintype`, `Set.Finite`, and decidable equality.

Analysis and library use - core, study weeks 15-26

- ☐ Interpret `Tendsto`, `atTop`, neighborhoods, filter order, `Eventually`, and `Frequently`.
- ☐ Translate an epsilon statement into a filter statement; compose limit and continuity lemmas.
- ☐ Explain why uniqueness of limits needs nontrivial-filter and separation assumptions.
- ☐ Handle `Finset.range`/intervals, finite-sum induction, casts, and endpoint cases.
- ☐ Distinguish `HasSum`, `Summable`, and `tsum`; recognize default unconditional summation.
- ☐ Use positivity, comparison, geometric series, and divisibility/factorial facts with their side conditions.
- ☐ Distinguish `HasDerivAt`, `DifferentiableAt`, and `deriv`; use a derivative theorem correctly.
- ☐ Navigate Mathlib by the needed statement; explain the inputs to a successful theorem application.

Broad analysis vocabulary - survey, study weeks 13-26

- ☐ Read compactness/extreme-value statements and the basic normed-space hierarchy.
- ☐ Recognize Frechet derivatives and continuous linear maps; distinguish from scalar derivatives.
- ☐ Read measurability, measures valued in extended nonnegative reals, and almost-everywhere claims.
- ☐ Distinguish integrals over a set, an oriented interval, and a measure; check integrability.
- ☐ Recognize the simple-function/L1 construction of the Bochner integral without reproducing it.

Verification and trust - core, introduced early and assessed in study week 28

- ☐ Explain elaboration/tactics versus kernel checking, and `#eval` versus a theorem.
- ☐ Inspect exact statements for vacuity, missing restrictions, total-function defaults, and wrong domains.
- ☐ Audit dependencies with `#print axioms`; detect `sorryAx` and unexpected custom axioms.
- ☐ Explain classical choice, propositional extensionality, quotients, proof irrelevance, and `noncomputable` at a basic level.
- ☐ Check every claimed file in the pinned environment and save a replay command and version information.

3. Practice, accountability, and completion

A sustainable three-and-a-half-hour week

- Monday, 50 minutes: 5-minute preview, 30-minute video segment, 15-minute reconstruction.
- Tuesday, 60 minutes: independent homework attempt with a paper argument first.
- Thursday, 60 minutes: continue the proof or finish the week's named intermediate milestone.
- Friday, 40 minutes: targeted reading/exercises, checking, delayed recall, and Progress.md.
- Optional, 30 minutes: a hard proof or checkpoint. The weekly total remains within four hours.

Keep roughly two thirds of your time for active proofs. Split long videos at topic boundaries. After each segment: one reconstructed example, one explained definition, one new variation. Record concepts and obstacles rather than a transcript.

Independent checkpoints

- **A, study week 6:** Six unfamiliar short proofs across logic, quantifiers, rewriting, and induction. Retry gaps; fewer than five successes in 60-90 minutes signals another foundations week.
- **B, study week 14:** A recursive definition and invariant, an extensional equality, and an explained instance. Choose the final mini-project.
- **C, study week 22:** A new elementary limit, a comparison/summability variation, and a finite-sum identity, using documentation but no generated solution.
- **D, study week 28:** A small formalization with 2-4 supporting lemmas, a checked main theorem, assumptions/dependency audit, replay instructions, and a 10-minute explanation.

Checkpoints replace normal practice blocks; they do not add hours. Reestimate the remaining schedule after the first two units. A missed week moves the deadline, not the amount of work into the next week.

A finished homework has five kinds of evidence

☐ The exact intended statement and definitions, with all required parts completed.
☐ That exact working file checks; all its warnings have been reviewed.
☐ No `sorry`/`admit` dependencies or unexplained custom axioms in claimed results.
☐ The original exercise's tactic and import restrictions are respected.
☐ You can explain the proof and reconstruct a representative part 48-72 hours later.

In Propositional and Quantifiers, follow the starter's tactic whitelist. Induction forbids `grind` and permits `omega`. Finish `#min_imports` cleanup where requested. A successful build or a printed score by itself does not demonstrate understanding.

The verification habit

Check the intended statement, then the proof and dependencies. Use `#check`, `#print`, and `#print axioms` for each named result. `sorryAx` is a gap; ordinary dependencies such as `propext`, `Classical.choice`, and `Quot.sound` are not automatically gaps. Review the assumptions under which a theorem is proved.

The lectures include deliberate errors, unfinished examples, and `axiom bad : 2 + 2 = 5`. Work in new practice files rather than importing entire lectures. Expressions for division, derivatives, series, and integrals can have default values outside the intended mathematical conditions: inspect side conditions.

Make the commitment small and visible

Every Friday, produce one checked file or a named partial milestone. Update Progress.md in two minutes. Arrange a fortnightly peer review or record a five-minute explanation. If stuck, classify the obstacle, write the exact goal, then seek one hint. Rebuild any helped proof independently after 48-72 hours. After a missed week, resume the next incomplete prerequisite and shift the dates.

Sources and scope: Repository snapshot 381add8, checked 21 October 2026. The detailed guide contains local file links and the chapter crosswalk. Main references: Mathematics in Lean and Theorem Proving in Lean 4; for trust, see TPiL: Axioms and Computation. Course pins: Lean/Mathlib v4.33.1. Timelines and checkpoints are recommendations; no course build or grading run was performed in preparing this guide.